



รายงานการวิเคราะห์และเปรียบเทียบอัลกอริทึม

Food Delivery Order Queue

รายวิชา

Data Structures and Algorithms

ผู้จัดทำ

นาย ณัฐนันท์ เอ่งฉ้วน 68122250001

นาย พุฒิชัย โพธิ์ขำ 68122250094

นาย ธนกร ชุมเดชา 68122250029

นาย ตะวัน นามลนนวน 67122250094

เสนอ

ผศ.ดร. กัลยณัฐ กุหลาบเพชรทอง

รายงานนี้เป็นส่วนหนึ่งของวิชา CSD2103

การออกแบบและวิเคราะห์ขั้นตอนวิธี

สาขาวิชาวิทยาการคอมพิวเตอร์และนวัตกรรมข้อมูล

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏสุรินทร์

ภาคเรียนที่ 1 ปีการศึกษา 2569

สารบัญ

บทที่ 1 บทนำ

- 1.1 ที่มาและความสำคัญ
- 1.2 Case Story: Food Delivery Order Queue
- 1.3 วัตถุประสงค์
- 1.4 ขอบเขตงาน

บทที่ 2 การวิเคราะห์ปัญหา

- 2.1 Input
- 2.2 Output
- 2.3 Constraints
- 2.4 Assumptions
- 2.5 Edge Cases

บทที่ 3 การออกแบบอัลกอริทึม

- 3.1 Queue Design
- 3.2 Algorithm A: FIFO Queue
- 3.3 Algorithm B: Priority Queue
- 3.4 Pseudocode Algorithm A
- 3.5 Pseudocode Algorithm B
- 3.6 Queue Trace 10 Operations
- 3.7 Flowchart
- 3.8 Step-by-step การทำงาน

บทที่ 4 การอธิบายความถูกต้อง

- 4.1 Precondition
- 4.2 Correctness / Invariant
- 4.3 Postcondition

4.4 Termination

บทที่ 5 การวิเคราะห์ประสิทธิภาพ

5.1 Best Case

5.2 Average Case

5.3 Worst Case

5.4 Time Complexity

5.5 Space Complexity

5.6 Trade-offs

บทที่ 6 การพัฒนาโปรแกรม Java

6.1 Class Diagram

6.2 โครงสร้างคลาส

6.3 เมธอดสำคัญ

6.4 Input Validation

6.5 Error Handling

บทที่ 7 การทดลอง

7.1 Test Cases 6 กรณี

7.2 Experimental Setup

7.3 Queue Scenario Waiting Time

7.4 ตารางผล Benchmark

7.5 กราฟ

7.6 การอภิปรายผล

บทที่ 8 สรุป

8.1 วิธีที่เหมาะสมกว่า

8.2 เหตุผลในการเลือก

8.3 ข้อจำกัด

8.4 แนวทางพัฒนาต่อ

8.5 สิ่งที่ได้เรียนรู้

8.6 การใช้ Generative AI

บรรณานุกรม / แหล่งข้อมูล

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

ระบบสั่งอาหารผ่านแพลตฟอร์ม Food Delivery สามารถสร้าง Order หลายรายการเข้าสู่ร้านอาหารในช่วงเวลาใกล้เคียงกัน ร้านจึงต้องกำหนดลำดับในการนำ Order ไปเตรียมอาหารเพื่อให้การทำงานมีประสิทธิภาพและสามารถให้บริการลูกค้าได้อย่างเหมาะสม ปัญหานี้เป็นตัวอย่างที่ตรงกับการประยุกต์ใช้โครงสร้างข้อมูล Queue ซึ่งมีหน้าที่เก็บรายการที่กำลังรอการประมวลผล

โครงงานนี้จะออกแบบและเปรียบเทียบการจัดคิว 2 วิธี ได้แก่ Algorithm A: FIFO Queue ซึ่งให้บริการตามลำดับการเข้ามาของ Order และ Algorithm B: Priority Queue ซึ่งให้บริการ Order ที่มี Priority สูงกว่าก่อน โดยในโค้ดกำหนด Priority เป็น 1 = Express, 2 = Normal และ 3 = Large หาก Priority เท่ากันจะพิจารณา orderTime เพื่อรักษาลำดับของ Order ที่มีความสำคัญเท่ากัน

ประเด็นที่นำมาเปรียบเทียบประกอบด้วย Waiting Time, Fairness และ Execution Time เพื่อให้เห็นทั้งมุมมองของผู้ใช้บริการและมุมมองด้านประสิทธิภาพของโปรแกรม โดยข้อมูลและแนวทางการทดลองยึดตามโปรเจกต์ Java ของกลุ่ม 5 ใน Repository FoodDeliveryQueue_Group5

1.2 Case Story: Food Delivery Order Queue

สมมติว่าร้านอาหารได้รับ Order จาก Food Delivery Platform จำนวน 5 รายการ โดยแต่ละรายการมีเวลาที่เข้าระบบ ประเภทความเร่งด่วน และเวลาที่ใช้ในการเตรียมแตกต่างกัน ดังนี้

Order ID	Order Time	Food Type	Preparation Time	Priority
O1	1	Normal Food	15 นาที	Normal (2)
O2	2	Express Food	5 นาที	Express (1)
O3	3	Normal Food	10 นาที	Normal (2)
O4	4	Express Food	8 นาที	Express (1)
O5	5	Large Food	25 นาที	Large (3)

สำหรับ Algorithm A ลำดับที่ได้คือ O1 -> O2 -> O3 -> O4 -> O5 ส่วน Algorithm B ลำดับที่ได้คือ O2 -> O4 -> O1 -> O3 -> O5 ซึ่งสะท้อนความแตกต่างระหว่างการเน้นความยุติธรรมตามลำดับเวลาและการเน้นงานเร่งด่วน

1.3 วัตถุประสงค์

- ศึกษาและประยุกต์ใช้โครงสร้างข้อมูล Queue กับปัญหาการจัดลำดับ Order
- ออกแบบ Algorithm A และ Algorithm B สำหรับเปรียบเทียบการจัดคิว 2 รูปแบบ
- วิเคราะห์ Waiting Time และ Fairness ของแต่ละ Algorithm
- วิเคราะห์ Correctness, Invariant, Time Complexity และ Space Complexity
- พัฒนาโปรแกรมภาษา Java และจัดทำ Test Cases อย่างน้อย 6 กรณี
- ทดลองเปรียบเทียบ Execution Time ที่ขนาดข้อมูล 100, 1,000, 10,000 และ 50,000 รายการ

1.4 ขอบเขตงาน

- ใช้ข้อมูล Order แบบกำหนดค่าไว้ในโปรแกรมเพื่อสร้าง Scenario และทดสอบ
- ใช้ ArrayDeque สำหรับ FIFO Queue และ PriorityQueue สำหรับ Priority Queue
- เปรียบเทียบลำดับการประมวลผล Waiting Time, Maximum Waiting Time และ Average Waiting Time
- มี Queue Trace จำนวน 10 Operations
- มี Test Cases จำนวน 6 กรณี ได้แก่ Normal, Empty, Single Item, Large Queue, Same Priority และ Cancel
- Benchmark ใช้ข้อมูล $n = 100, 1,000, 10,000, 50,000$ และทดลองซ้ำ 5 รอบตามเอกสารผลการทดลองของโครงการ

บทที่ 2 การวิเคราะห์ปัญหา

2.1 Input

ข้อมูลนำเข้าสำหรับแต่ละ Order ถูกแทนด้วย Order Object ซึ่งประกอบด้วยข้อมูลหลัก 5 รายการ ได้แก่ Order ID, Order Time, Food Type, Preparation Time และ Priority โดย Priority ใช้ตัวเลข 1, 2, 3 เพื่อให้ Comparator ของ PriorityQueue สามารถจัดลำดับได้โดยตรง

Field	ชนิดข้อมูล	ความหมาย
id	String	รหัส Order เช่น O1, O2
orderTime	int	ลำดับ/เวลาที่ Order เข้ามา
type	String	ประเภทอาหาร
prepTime	int	เวลาที่ใช้เตรียมอาหารเป็นนาที
priority	int	1=Express, 2=Normal, 3=Large

2.2 Output

- ลำดับ Order ที่ถูกนำไปประมวลผล
- Waiting Time ของแต่ละ Order
- Average Waiting Time
- Maximum Waiting Time
- ผลการทดสอบ Queue Operations
- ผลการ Benchmark Execution Time ของ Algorithm A และ B

2.3 Constraints

- FIFO ต้องรักษาลำดับการเข้ามาของ Order
- Priority Queue ต้องเลือก Priority ที่สูงกว่า โดยตัวเลข Priority ที่น้อยกว่าหมายถึงความสำคัญสูงกว่า
- หาก Priority เท่ากัน ให้ใช้ orderTime เป็นตัวตัดสิน
- Preparation Time ของแต่ละ Order อาจแตกต่างกัน

- ต้องรองรับ Queue ว่างและข้อมูลจำนวนน้อยหรือมาก
- การยกเลิก Order ในตัวอย่างใช้ `removeIf` และ Order ที่ถูกลบต้องไม่คงอยู่ใน Queue

2.4 Assumptions

- สมมติว่าร้านอาหารประมวลผล Order ทีละรายการ
- `currentTime` เริ่มต้นที่ 0 นาฬิกา
- Waiting Time ของ Order เท่ากับ `currentTime` ณ เวลาที่ Order ถูกนำออกจาก Queue
- Preparation Time เป็นเวลาที่ใช้ในการเตรียมอาหารและเพิ่มเข้า `currentTime` หลังประมวลผล
- การเปรียบเทียบ Priority ใช้กฎเดียวกันตลอดการทดลอง
- Benchmark วัดเวลาช่วงการสร้างและประมวลผล Queue ด้วย `System.nanoTime()`

2.5 Edge Cases

กรณี	สิ่งที่ต้องตรวจสอบ	ผลที่คาดหวัง
Queue ว่าง	<code>poll()</code> และ <code>peek()</code>	คืนค่า null โดยไม่ทำให้โปรแกรมล้ม
มี 1 Order	<code>enqueue</code> แล้ว <code>dequeue</code>	ได้ Order เดิมกลับออกมา
Priority เท่ากัน	Comparator ใช้ <code>orderTime</code> ต่อ	Order ที่เข้าก่อนมาก่อน
Large Queue	เพิ่ม 10,000 Orders	Queue มีขนาด 10,000 ได้
Cancel	<code>removeIf</code> ตาม Order ID	Order ที่ยกเลิกถูกลบออก
หลาย Priority	Express/Normal/Large	Express มาก่อน Normal มาก่อน Large เมื่อ <code>orderTime</code> ไม่ขัดกฎ

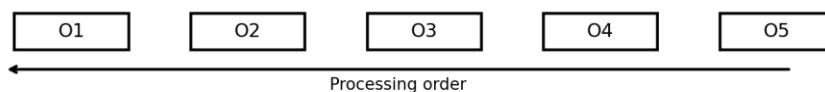
บทที่ 3 การออกแบบอัลกอริทึม

3.1 Queue Design

ระบบเลือกใช้ Queue 2 implementation เพื่อให้เห็นข้อแตกต่างระหว่างการจัดคิวแบบ FIFO และการจัดคิวแบบ Priority โดย ArrayDeque เหมาะกับการเพิ่มและนำข้อมูลออกที่ปลาย Queue ส่วน PriorityQueue ใช้ตัวเปรียบเทียบเพื่อเลือก Order ที่มี Priority สูงสุดก่อน

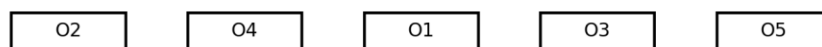
Algorithm A: FIFO

Arrival order -> process order



Algorithm B: Priority Queue

Higher priority first; tie -> orderTime



รูปที่ 3.1 การออกแบบ Queue สำหรับ Algorithm A และ Algorithm B

ประเด็น	Algorithm A: FIFO	Algorithm B: Priority Queue
Implementation	ArrayDeque	PriorityQueue
กฎการเลือก	Order ที่เข้าก่อนถูกทำก่อน	Priority สูงก่อน; ถ้าเท่ากันใช้ orderTime
จุดเด่น	ง่าย รวดเร็ว และ Fairness สูง	ลดเวลารอของ Express ได้
ข้อจำกัด	Express อาจต้องรอ Order ก่อนหน้า	อาจเกิด Starvation กับ Priority ต่ำ
ต่อ Operation	Enqueue/Dequeue โดยทั่วไป $O(1)$	Offer/Poll โดยทั่วไป $O(\log n)$

3.2 Algorithm A: FIFO Queue

Algorithm A ใช้ Queue<Order> ที่สร้างจาก ArrayDeque เมื่อสร้าง Scenario แล้วโปรแกรมใช้ addAll() เพื่อเติม Order ตามลำดับ จากนั้น processQueue() จะ poll() Order จากด้านหน้าและคำนวณ Waiting Time โดย currentTime ถูกเพิ่มด้วย prepTime ของ Order ที่เพิ่งประมวลผล

```
Queue<Order> queue = new ArrayDeque<>();
queue.addAll(orders);

while (!queue.isEmpty()) {
    Order o = queue.poll();
    int waiting = currentTime;
    currentTime += o.prepTime;
}
```

3.3 Algorithm B: Priority Queue

Algorithm B ใช้ PriorityQueue ร่วมกับ Comparator ที่เรียง priority จากน้อยไปมาก และเรียง orderTime จากน้อยไปมากเมื่อ Priority เท่ากัน ดังนั้น Priority 1 (Express) ถูกเลือกก่อน Priority 2 (Normal) และ Priority 3 (Large)

```
PriorityQueue<Order> queue = new PriorityQueue<>(
    Comparator.comparingInt((Order o) -> o.priority)
                .thenComparingInt(o -> o.orderTime)
);
queue.addAll(orders);

while (!queue.isEmpty()) {
    Order o = queue.poll();
    int waiting = currentTime;
    currentTime += o.prepTime;
}
```

3.4 Pseudocode Algorithm A

ALGORITHM A: FIFO QUEUE

1. Create an empty FIFO queue
2. Insert every Order in arrival order
3. Set $currentTime = 0$
4. While queue is not empty
 - 4.1 Remove the front Order
 - 4.2 $waitingTime = currentTime$
 - 4.3 Add $waitingTime$ to $totalWaiting$
 - 4.4 $currentTime = currentTime + prepTime$
5. Calculate average waiting time
6. Return processing result

3.5 Pseudocode Algorithm B

ALGORITHM B: PRIORITY QUEUE

1. Create an empty priority queue
2. Insert every Order using comparator (priority, orderTime)
3. Set $currentTime = 0$
4. While queue is not empty
 - 4.1 Remove the highest-priority Order
 - 4.2 $waitingTime = currentTime$
 - 4.3 Add $waitingTime$ to $totalWaiting$
 - 4.4 $currentTime = currentTime + prepTime$
5. Calculate average waiting time
6. Return processing result

3.6 Queue Trace 10 Operations

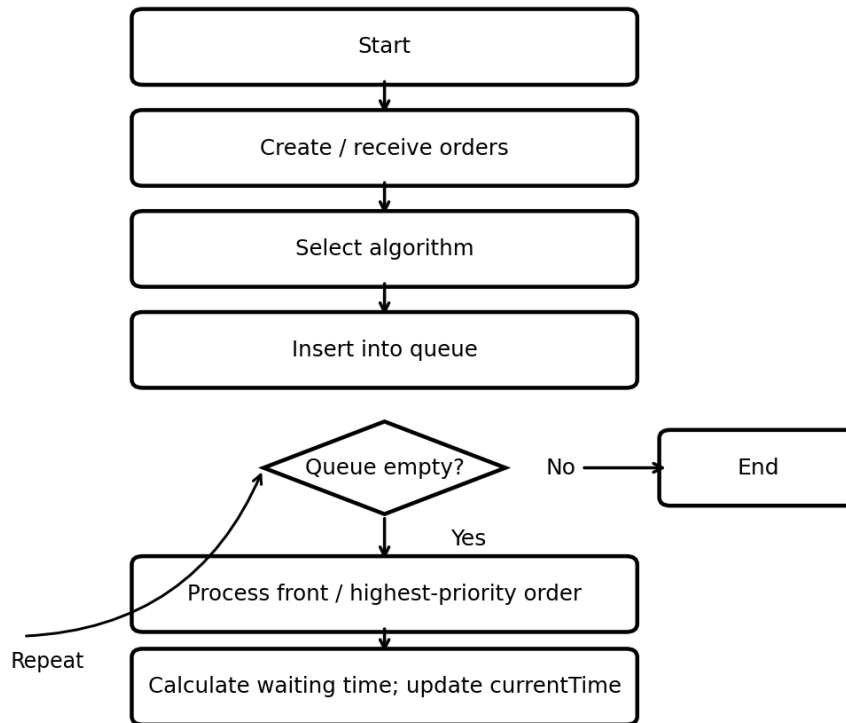
โปรแกรมมีการสาธิต Queue Trace จำนวน 10 Operations เพื่อแสดงสถานะ Queue ก่อนและหลังการทำงาน

Step	Operation	Queue Before	Queue After	Output
1	ENQUEUE O1	[]	[O1]	-
2	ENQUEUE O2	[O1]	[O1, O2]	-
3	ENQUEUE O3	[O1, O2]	[O1, O2, O3]	-
4	PEEK	[O1, O2, O3]	[O1, O2, O3]	O1
5	DEQUEUE	[O1, O2, O3]	[O2, O3]	O1
6	ENQUEUE O4	[O2, O3]	[O2, O3, O4]	-
7	ENQUEUE O5	[O2, O3, O4]	[O2, O3, O4, O5]	-
8	PEEK	[O2, O3, O4, O5]	[O2, O3, O4, O5]	O2
9	DEQUEUE	[O2, O3, O4, O5]	[O3, O4, O5]	O2
10	DEQUEUE	[O3, O4, O5]	[O4, O5]	O3

ตารางที่ 3.1 Queue Trace จำนวน 10 Operations

3.7 Flowchart

Flowchart: Processing an Order



รูปที่ 3.2 Flowchart การประมวลผล Order

3.8 Step-by-step การทำงาน

- สร้างข้อมูล Scenario ของ Order 5 รายการ
- เลือก Algorithm A หรือ Algorithm B
- นำ Order เข้าสู่ Queue ตามกฎของ Algorithm
- ตรวจสอบ Queue ว่ายังมี Order หรือไม่
- นำ Order ที่ถึงลำดับออกจาก Queue ด้วย poll()
- คำนวณ Waiting Time จาก currentTime
- เพิ่ม Preparation Time เข้า currentTime
- ทำซ้ำจน Queue ว่าง แล้วคำนวณ Average และ Maximum Waiting Time

บทที่ 4 การอธิบายความถูกต้อง

4.1 Precondition

- Queue ถูกสร้างและพร้อมใช้งาน
- ข้อมูล Order มีค่า id, orderTime, prepTime และ priority ตามชนิดข้อมูลที่กำหนด
- currentTime เริ่มต้นที่ 0
- Comparator ของ PriorityQueue ใช้ priority และ orderTime ตามกติกา

4.2 Correctness / Invariant

Invariant ของ Algorithm A คือ เมื่อมี Order A ถูกใส่เข้าคิวก่อน Order B หากทั้งสองยังไม่ถูกประมวลผล A จะอยู่ก่อน B ใน FIFO Queue เสมอ ดังนั้นการใช้ poll() จะนำ A ออกก่อน B และไม่เปลี่ยนลำดับของข้อมูลที่ยังรออยู่

Invariant ของ Algorithm B คือ Order ที่อยู่ต้น Queue จะเป็น Order ที่มีลำดับตาม Comparator ที่ถูกต้อง กล่าวคือ priority ที่มีค่าน้อยกว่าจะมาก่อน และเมื่อ Priority เท่ากัน orderTime ที่น้อยกว่าจะมาก่อน จึงสอดคล้องกับกติกาการจัดคิวที่กำหนด

Waiting Time Invariant: ณ จุดเริ่มต้นของการประมวลผล Order แต่ละตัว currentTime เท่ากับผลรวมของ prepTime ของ Order ที่ประมวลผลเสร็จก่อนหน้านี้ ดังนั้น $waitingTime = currentTime$ ให้เวลารอที่ถูกต้องภายใต้สมมติฐานว่าร้านอาหารเตรียม Order ที่ละรายการ

4.3 Postcondition

- เมื่อ processQueue() จบ Queue ต้องว่าง
- totalWaiting เท่ากับผลรวม Waiting Time ของ Order ทั้งหมด
- $average = totalWaiting / \text{จำนวน Order}$ เมื่อจำนวน Order มากกว่า 0
- maxWaiting เท่ากับ Waiting Time ที่มากที่สุด
- Order ทุกตัวถูกประมวลผลตามกฎหมายของ Algorithm ที่เลือก

4.4 Termination

ในแต่ละรอบของ while loop จะมีการ poll() Order ออกจาก Queue และขนาด Queue ลดลงอย่างน้อย 1 รายการ เมื่อมีจำนวน Order จำกัด การวนซ้ำจึงสิ้นสุดเมื่อ queue.isEmpty() เป็นจริง จึงรับประกันการสิ้นสุดของ processQueue() สำหรับข้อมูลที่ถูกต้อง

บทที่ 5 การวิเคราะห์ประสิทธิภาพ

5.1 Best Case

สำหรับ Algorithm A การ enqueue และ dequeue ของ ArrayDeque เป็น $O(1)$ โดยทั่วไปต่อ Operation เมื่อพิจารณา n Operations จึงมีเวลาโดยรวมเป็น $O(n)$ ส่วน Algorithm B ต้องรักษา heap order ทำให้การแทรกและนำออกมี $O(\log n)$ ต่อ Operation และเมื่อมี n รายการจะได้ $O(n \log n)$ ในภาพรวม

5.2 Average Case

กรณีเฉลี่ยของ FIFO ยังคงเติบโตเชิงเส้นตามจำนวน Order เพราะแต่ละ Order ถูกเพิ่มและนำออกครั้งเดียว สำหรับ Priority Queue จำนวนครั้งของการจัดโครงสร้าง heap ทำให้เวลาเฉลี่ยโดยรวมเพิ่มเร็วกว่า FIFO เมื่อ n มีค่ามากขึ้น

5.3 Worst Case

FIFO: Enqueue/Dequeue เป็น $O(1)$ ต่อ Operation โดยทั่วไป ดังนั้น n Operations ใช้ $O(n)$ เวลา ส่วน Priority Queue: Offer และ Poll มีค่า $O(\log n)$ ในกรณีทั่วไป/กรณีเลวร้ายตามการปรับ heap ดังนั้นการประมวลผล n รายการอยู่ที่ $O(n \log n)$ โดยรวม
ทั้งนี้การวิเคราะห์เชิงทฤษฎีแยกจากเวลารันจริงซึ่งยังขึ้นกับ JVM, CPU และการจัดสรรหน่วยความจำ

5.4 Time Complexity

Operation	Algorithm A: ArrayDeque	Algorithm B: PriorityQueue
offer / add	$O(1)$ โดยทั่วไป	$O(\log n)$
poll	$O(1)$ โดยทั่วไป	$O(\log n)$
peek	$O(1)$	$O(1)$
ประมวลผล n Orders	$O(n)$	$O(n \log n)$
Cancel ด้วย removeIf	$O(n)$	$O(n)$ โดยการค้นหา/ลบตามเงื่อนไข

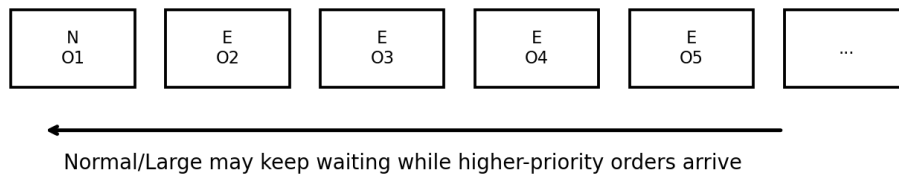
5.5 Space Complexity

ทั้งสอง Algorithm เก็บ Order จำนวน n รายการไว้ใน Queue จึงมี Space Complexity เป็น $O(n)$ สำหรับข้อมูลที่จัดเก็บ ส่วนตัวแปรประกอบการคำนวณ เช่น `currentTime`, `totalWaiting` และ `maxWaiting` ใช้พื้นที่คงที่ $O(1)$ ไม่ว่าจะมีย Order จำนวนเท่าใด

5.6 Trade-offs

ด้านเปรียบเทียบ	FIFO	Priority Queue
Waiting Time	20.60 นาทีใน Scenario	16.80 นาทีใน Scenario
Fairness	สูงกว่า เพราะไม่แซงคิว	ต่ำกว่า เพราะ Priority สูงแซงได้
Execution Time	เร็วกว่าเมื่อ n ใหญ่	ช้ากว่าเนื่องจาก heap overhead
Use case	งานที่ต้องการความยุติธรรม	งานที่มี Order เร่งด่วน
Risk	Express รอนานได้	Normal/Large อาจ Starvation

Priority Queue: Starvation Risk

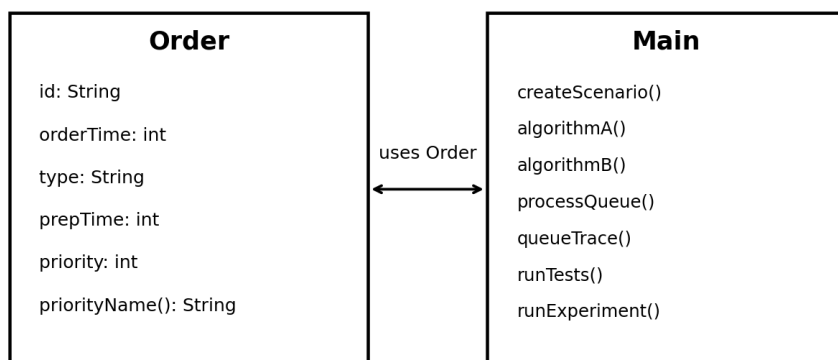


รูปที่ 5.1 ตัวอย่างความเสี่ยง Starvation ของ Priority Queue

บทที่ 6 การพัฒนาโปรแกรม Java

6.1 Class Diagram

Class Diagram (Simplified)



รูปที่ 6.1 Class Diagram แบบย่อของโปรแกรม

6.2 โครงสร้างคลาส

องค์ประกอบ	หน้าที่
Order	เก็บข้อมูล id, orderTime, type, prepTime และ priority
Main	รวมเมธอดสำหรับสร้าง Scenario, รัน Algorithm A/B, Trace, Test และ Experiment
ArrayDeque<Order>	ใช้เป็น FIFO Queue
PriorityQueue<Order>	ใช้เป็น Priority Queue และ Comparator เป็นตัวกำหนดลำดับ

6.3 เมธอดสำคัญ

เมธอด	หน้าที่
createScenario()	สร้าง Order ตัวอย่าง 5 รายการ
algorithmA()	เตรียม ArrayDeque และเรียก processQueue()
algorithmB()	เตรียม PriorityQueue พร้อม Comparator
processQueue()	poll Order, คำนวณ Waiting Time และสรุปผล
queueTrace()	สาธิต Queue Operations 10 ขั้นตอน
runTests()	ทดสอบ 6 กรณี
runExperiment()	วัด Execution Time ที่ n หลายขนาด

6.4 Input Validation

โปรแกรมใน Repository เป็นโปรแกรมสาธิตที่สร้างข้อมูลจาก createScenario() และกำหนด Test Cases ไว้ในโค้ด จึงไม่มีส่วนรับ Input จากผู้ใช้แบบ Interactive และไม่มีเมธอด validation ที่ซับซ้อน การควบคุมข้อมูลจึงอาศัยการสร้าง Order ด้วยชนิดข้อมูลที่กำหนดไว้

6.5 Error Handling

- Queue ว่าง: poll() และ peek() ของ Queue ที่สร้างด้วย ArrayDeque ใน Test Case คืน null
- Priority Queue: หาก Priority เท่ากัน Comparator ใช้ orderTime เป็นตัวตัดสิน
- Cancel: ใช้ removeIf และตรวจสอบค่าที่คืนมาว่าลบสำเร็จหรือไม่
- ข้อมูล Scenario ถูกกำหนดล่วงหน้าเพื่อลดความคลาดเคลื่อนในการเปรียบเทียบ

บทที่ 7 การทดลอง

7.1 Test Cases 6 กรณี

TC	กรณีทดสอบ	การทำงาน	ผลที่คาดหวัง
1	Normal Case	เพิ่ม O1 และ O2 ใน FIFO	Queue เก็บ O1, O2 ตามลำดับ
2	Empty Queue	poll และ peek จาก Queue ว่าง	ได้ null ทั้งคู่
3	Single Item	เพิ่ม O1 แล้ว poll	ได้ O1
4	Large Queue	เพิ่ม 10,000 Orders	size = 10,000
5	Same Priority	O1,O2,O3 Priority เท่ากัน	เรียง O1 O2 O3 ตาม orderTime
6	Cancel Case	removelf ลบ O2	O2 ถูกลบ เหลือ O1 และ O3

ตารางที่ 7.1 Test Cases อย่างน้อย 6 กรณี

7.2 Experimental Setup

โค้ด runExperiment() กำหนดขนาดข้อมูล 100, 1,000, 10,000 และ 50,000 รายการ ใช้ seed = 12345L และ warm-up 3 รอบก่อนวัดจริง 5 รอบ โดยใช้ System.nanoTime() สำหรับการจับเวลา ตัวเลขในตารางต่อไปนี้ยึดตามเอกสาร Experimental Comparison ที่จัดเก็บอยู่ใน Repository ของโครงการ

Parameter	ค่าที่ใช้
Input size (n)	100, 1,000, 10,000, 50,000
Warm-up	3 รอบ
Measurement	5 รอบ
Random seed	12345L
Timer	System.nanoTime()
Metric	Average execution time

7.3 Queue Scenario Waiting Time

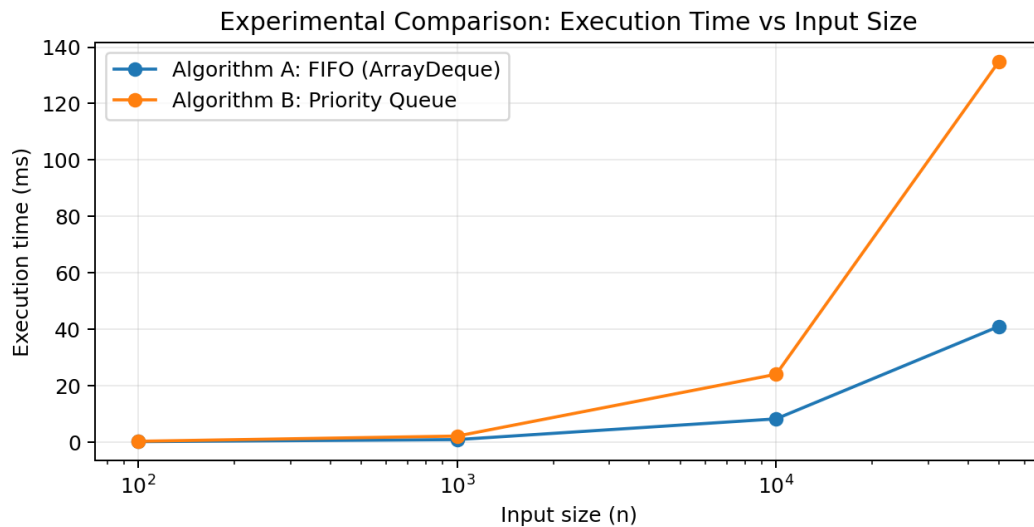
Algorithm	Processing Order	Waiting Time	Average	Maximum
A: FIFO	O1 -> O2 -> O3 -> O4 -> O5	0, 15, 20, 30, 38	20.60 นาที	38 นาที
B: Priority	O2 -> O4 -> O1 -> O3 -> O5	0, 5, 13, 28, 38	16.80 นาที	38 นาที

Priority Queue ลด Average Waiting Time จาก 20.60 นาทีเหลือ 16.80 นาที
คิดเป็นการลดลงประมาณ 18.45% ใน Scenario นี้ ขณะที่ Maximum Waiting Time ยังคงเท่ากันที่
38 นาที

7.4 ตารางผล Benchmark

n	FIFO avg (ms)	Priority avg (ms)	Priority ช้ากว่าโดยประมาณ
100	0.12	0.25	2.08 เท่า
1,000	0.85	2.10	2.47 เท่า
10,000	8.20	24.00	2.93 เท่า
50,000	41.00	135.00	3.29 เท่า

7.5 กราฟ



รูปที่ 7.1 กราฟเปรียบเทียบ Execution Time กับ Input Size

7.6 การอภิปรายผล

ผลการทดลองที่บันทึกในโครงการแสดงแนวโน้มว่า FIFO ใช้เวลาน้อยกว่า Priority Queue ในทุกขนาดข้อมูลที่ทดสอบ และส่วนต่างเพิ่มขึ้นเมื่อ n เพิ่มขึ้น สอดคล้องกับความแตกต่างด้าน Time Complexity คือ FIFO มีต้นทุน $O(1)$ ต่อ Queue Operation ขณะที่ Priority Queue มีต้นทุน $O(\log n)$ ต่อการแทรก/นำออก

ในมุมมองของผู้ใช้บริการ Priority Queue ให้ผลดีกว่าใน Scenario เพราะ Express Order ได้รับการเตรียมก่อน ทำให้ Average Waiting Time ต่ำกว่า อย่างไรก็ตามการจัดลำดับด้วย Priority ทำให้เกิด trade-off ด้าน Fairness และอาจเกิด Starvation หาก Priority สูงเข้ามาอย่างต่อเนื่อง

หมายเหตุ: Execution Time เป็นผลจาก benchmark ที่บันทึกใน Repository และไม่ควรตีความเป็นค่าคงที่สำหรับคอมพิวเตอร์ทุกเครื่อง เนื่องจาก CPU, JVM, memory และสภาพแวดล้อมมีผลต่อเวลารันจริง

บทที่ 8 สรุป

8.1 วิธีที่เหมาะสมกว่า

ไม่มี Algorithm ใดดีกว่าในทุกมิติ หากเกณฑ์หลักคือความยุติธรรมและความง่ายในการประมวลผล FIFO เป็นทางเลือกที่เหมาะสมกว่า แต่ถ้าวัดระบบให้ความสำคัญกับ Order เร่งด่วนและต้องการลดเวลารอของ Express Priority Queue เหมาะสมกว่า

8.2 เหตุผลในการเลือก

- FIFO: โครงสร้างง่ายและเร็ว เหมาะกับระบบที่ต้องรักษาลำดับมาก่อน-ทำก่อน
- Priority Queue: รองรับความแตกต่างของความเร่งด่วนและลด Average Waiting Time ของ Express ใน Scenario
- เมื่อจำนวนข้อมูลเพิ่มขึ้น Priority Queue มี Execution Time สูงกว่า แต่แลกกับความสามารถในการเลือกงานตาม Priority

8.3 ข้อจำกัด

- Scenario ที่ใช้เปรียบเทียบมี Order เพียง 5 รายการ
- Benchmark เวลาในเอกสารอ้างอิงขึ้นกับสภาพแวดล้อมของเครื่องที่ใช้ทดลอง
- โปรแกรมยังไม่มีส่วนติดต่อผู้ใช้และไม่มีการรับข้อมูลจากระบบภายนอก
- กฎ Priority ยังเป็นรูปแบบคงที่ 3 ระดับ และยังไม่มี Aging หรือการป้องกัน Starvation อัตโนมัติ

8.4 แนวทางพัฒนาต่อ

- เพิ่มระบบรับ Order จากผู้ใช้หรือไฟล์ข้อมูล
- เพิ่ม Aging เพื่อแก้ Starvation ของ Order Priority ต่ำ
- เพิ่มฟังก์ชัน Cancel และ Update Priority เป็นเมธอดของระบบโดยตรง
- เพิ่มการทดลองหลาย Scenario เช่น Express เข้ามาต่อเนื่อง หรือมีจำนวน Order หลายแสนรายการ
- เพิ่มการวัด Median และ Standard Deviation เพื่ออธิบายความผันผวนของ Execution Time
- พัฒนาเป็นระบบจำลองร้านอาหารที่มีหลายครัวหรือหลายช่องประมวลผล

8.5 สิ่งที่ได้เรียนรู้

โครงการทำให้เข้าใจการประยุกต์ใช้ Queue กับปัญหาจริง การเลือก implementation ให้เหมาะกับกฎการจัดคิว การเขียน Pseudocode การพิสูจน์ Correctness และ Invariant การวิเคราะห์ Time/Space Complexity ตลอดจนการออกแบบ Test Cases และ Benchmark เพื่อเปรียบเทียบ Algorithm อย่างเป็นระบบ

8.6 การใช้ Generative AI

ในการจัดทำเอกสารฉบับนี้มีการใช้ Generative AI เพื่อช่วยจัดโครงสร้างรายงาน เรียบเรียงคำอธิบายจากข้อมูลใน Repository ตรวจสอบความสอดคล้องของหัวข้อ และจัดรูปแบบเนื้อหาให้อยู่ในลักษณะรายงานวิชาการ โดยเนื้อหาหลักของ Algorithm, Scenario, Test Cases และผลการทดลองยึดจากไฟล์และโค้ดที่อยู่ใน Repository ของกลุ่ม

Repository ของโครงการ: https://github.com/nattanankao/FoodDeliveryQueue_Group5

บรรณานุกรม / แหล่งข้อมูล

- FoodDeliveryQueue_Group5, README.md และ README_หัวข้อวิเคราะห์เปรียบเทียบ.md, Repository ของกลุ่ม 5
- FoodDeliveryQueue_Group5, Main.java, โปรแกรม Java สำหรับ Queue, Test Cases และ Benchmark
- เอกสาร Problem Analysis.pdf, Queue Design.pdf, Pseudocode Algorithm A.pdf, Pseudocode Algorithm B.pdf, Test Cases ≥ 6.pdf, Time Complexity.pdf และ Experimental Comparison.pdf ใน Repository